

Demonstração e Apresentação de Desempenho de Modelos de Difusão de Contaminantes em Água

Fernando Daniel Marcelino¹, Lucas Moura da Silva¹,
Marcos Vinicius Gasparoto Bauab¹

¹Universidade Federal de São Paulo – Campus São José dos Campos
Avenida Cesare Mansueto Giulio Lattes, 1201 -Eugênio de Mello
São José dos Campos/SP - CEP: 12247-014

{fernando.marcelino, lucas.silva16, marcos.bauab}@unifesp.br

Abstract. *Water contamination by pollutants is a critical environmental issue that threatens global health and biodiversity, especially in poorer and developing countries. The diffusion of contaminants in water bodies is a complex phenomenon that can be modeled using partial differential equations. This study presents an approximate model for contaminant diffusion in water, implemented in the C programming language and parallelized using NVIDIA's CUDA framework. Performance tests were conducted to evaluate the impact of parallelization on the model's execution time. The results demonstrated that parallelization was highly effective in significantly reducing execution time, highlighting the relevance of GPU-based parallel architectures for addressing environmental problems requiring high computational power.*

Resumo. *A contaminação da água por poluentes é um problema ambiental crítico para a saúde e a biodiversidade global, especialmente nos países mais pobres e em desenvolvimento. A difusão de contaminantes em corpos d'água é um fenômeno complexo que pode ser modelado por equações diferenciais parciais. Neste trabalho, é apresentado um modelo aproximado de difusão de contaminantes em água, implementado na linguagem C e paralelizado utilizando o CUDA da NVIDIA. Foram realizados testes de desempenho para avaliar o impacto da paralelização no tempo de execução do modelo, e os resultados obtidos mostraram que a paralelização foi extremamente eficaz na redução do tempo de execução. Assim, destaca-se a relevância do uso de arquiteturas paralelas baseadas em GPUs para a solução de problemas ambientais que demandam alto poder computacional.*

1. Introdução

2. Fundamentação Teórica

A fundamentação teórica aborda as principais ferramentas utilizadas na implementação do modelo matemático realizado utilizando a programação paralela em *Compute Unified Device Architecture*(CUDA).

2.1. CUDA

A computação convencional que utiliza a CPU é baseada em um modelo de processamento sequencial, onde as instruções são executadas segundo uma ordem predefinida.

Contudo, esse tipo de abordagem começou a ser complementado há medida que a demanda por processamento de dados e instruções aumentaram, necessitando de uma quebra de paradigma de programação. Com o passar dos anos, esse novo paradigma de programação, começou a ser empregada nas áreas de processamento de imagens, simulações físicas, aprendizado de máquina e outras áreas que necessitam de um grande poder de processamento. E foi assim que começaram os estudos e desenvolvimento de tecnologias de processamento gráfico, as chamadas *Graphics Processing Units* (GPUs). Em meados da década de 1970 iniciou-se o desenvolvimento de tecnologias de processamento gráfico e um pouco antes dos anos 2000 conduziram-se estudos para a arquitetura usual de GPUs modernas. Com isso, o surgimento de arquiteturas heterogêneas GPU-CPU começaram a ser empregadas em aplicações de alto desempenho. [Cheng et al. 2014]

Com o aprimoramento das abordagens das GPUs, observou-se que elas poderiam ser utilizadas para programação de propósito geral, e não apenas para processamento gráfico. A NVIDIA, foi uma das pioneiras a desenvolver uma tecnologia de programação de propósito geral para GPUs, chamada de *Compute Unified Device Architecture* (CUDA). A tecnologia CUDA foi desenvolvida em 2006 e é uma plataforma de computação paralela que permite a programação de GPUs NVIDIA para aplicações de alto desempenho [NVIDIA Corporation 2012]. A programação em CUDA é feita em C/C++, e a plataforma oferece um conjunto de ferramentas, manuais e bibliotecas para facilitar o desenvolvimento de aplicações paralelas.

2.1.1. Núcleos de Processamento

O processamento com o CUDA é realizado através da implementação de um *kernel*, uma função especial criada na máquina CPU e executada na GPU. O *kernel* é executado por um grande número de *threads* em paralelo, e cada *thread* é executado por um núcleo de processamento da GPU. Os núcleos de processamento são organizados em blocos, e os blocos são organizados em *grids*. A utilização dos parâmetros do *kernel* é envolta de três “<<<” e “>>>”, onde o primeiro parâmetro é o número de blocos e o segundo é o número de *threads*.

```
1  __global__ void VecAdd(float* A, float* B, float* C)
2  {
3      int i = threadIdx.x;
4      C[i] = A[i] + B[i];
5  }
6
7  int main()
8  {
9      ...
10     // Kernel invocation with N threads
11     VecAdd<<<1, N>>>(A, B, C);
12     ...
13 }
```

Listagem 1. Exemplo de definição e invocação de um kernel CUDA para adição vetorial

O trecho de código apresentado na Listagem 1 mostra um exemplo de definição e invocação de um *kernel* CUDA para adição vetorial. Note que, para esse caso, terá-se apenas um bloco e, para cada bloco, *N threads*. Em seguida, o *kernel* recebe os parâmetros

da função a ser executada, que são três vetores de ponto flutuante A , B e C , e agora estará apto para realizar a adição dos elementos de A e B e armazenar o resultado em C . Nesse exemplo, o índice a se percorrer nos vetores é unidimensional, mas em outros problemas clássicos de programação paralela, como a multiplicação de matrizes, o índice pode ser bidimensional ou até mesmo tridimensional. Com isso, existe uma hierarquia nos blocos de *threads* e é necessário calcular os índices da *thread* para acessar os elementos dos vetores.

Um esquemático da organização dos blocos e *threads* pode ser observado na Figura 1, e o acesso à *thread* é feito através das funções *threadIdx.x*, *threadIdx.y* e *threadIdx.z*, juntamente com as funções *blockIdx.x*, *blockIdx.y* e *blockIdx.z* e, ainda, as funções *blockDim.x*, *blockDim.y* e *blockDim.z*. Dessa forma, tem-se o mapeamento completo de acesso às *threads* pela GPU. Para exemplificação, a Figura 1 exemplifica um esquema de identificação de *threads* e blocos em CUDA em uma grade de 2x2 blocos e 4x4 *threads*.

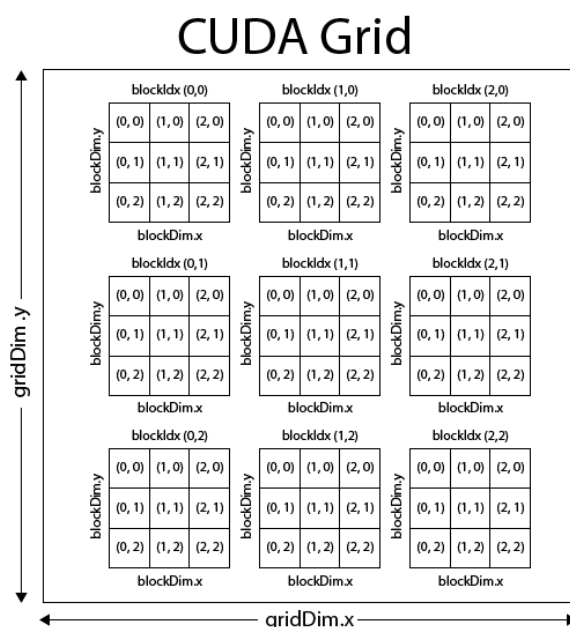


Figura 1. Esquema de identificação de threads e blocos em CUDA. [McKennon 2013]

2.2. Stencil

O stencil é amplamente usado em várias aplicações que utilizam a ciência computacional, como geofísica, astrofísica, física nuclear ou oceanográfica. Estas ciências necessitam de uma larga simulação computacional com uma frequência muito alta de saída. Quando é utilizado equações diferenciais parciais (EDP) normalmente elas são resolvidas pelo método das diferenças finitas, de modo a utilizar os stencil para calcular os operadores diferenciais representando uma grande fração do tempo total de execução.

Quando se está usando o stencil, para cada ponto, por exemplo de uma matriz 2D, é usado o peso dos pontos vizinhos, assim resolvendo os operadores diferenciais espacialmente, como mostra a Figura 2. Desta forma, replicando as operações para todos os pontos do conjunto, que ao intuito de agilizar os cálculos, pode ser feito uma paralelização

entre todos os pontos de modo ideal, fazendo com que cada *thread* um ponto para calcular, assim, possibilitando a paralelização do código. [Raúl de la Cruz and Araya-Polo 2015]

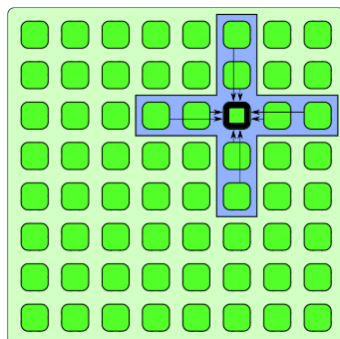


Figura 2. Exemplo de stencil 2D. [Stein and Griebler 2018]

2.3. GPU

A Unidade de Processamento Gráfico (*Graphics Processing Unit*, GPU) é um Processador especializado para o aceleração do processamento gráfico, especialmente em aplicações como renderização 3D, além do processamento de imagens. Com o avanço da arquiteturas computacionais, as GPUs evoluíram para plataformas de computação paralela de propósito geral, amplamente utilizadas em ciência de dados, aprendizado de máquina, simulações físicas e outras aplicações de alta performance, conhecidas como *General-Purpose Graphics Processing Unit* (GPGPU). [Owens et al. 2008]

Enquanto que as CPUs são otimizadas para a latência, uma GPU é otimizada para várias threads simultaneamente, dado a sua arquitetura massivamente paralela. Isso se dá ao fato de que a GPU é composta por centenas ou milhares de núcleos simplesmente organizados em múltiplos blocos, chamados *Streaming Multiprocessors* (SMs). Cada SM é projetado para executar centenas de threads em paralelo, com um modelo de execução baseado em *Single Instruction, Multiple Thread* (SIMT), uma das classificações de Flynn.

2.3.1. GPU T4

As GPUs T4 são baseadas na arquitetura Turing da NVIDIA, representam um avanço significativo no uso de GPUs para computação de propósito geral e aprendizado de máquina. A T4 foi projetada para ser uma solução otimizada para data centers modernos, suportando cargas de trabalho mistas que incluem treinamento, inferência, virtualização e serviços baseados em IA.

A GPU T4 tem uma memória compartilhada de 49KB, 4 MB de memória cache L2, Warp Size de 32 threads (Quantidade de threads que podem ser usadas simultaneamente). Os máximo de threads por bloco e multiprocessador é de 1024 e são 40 multiprocessadores, com uma dimensão máxima (1024, 1024, 64) de threads por bloco, em (x, y, z), já o grid é de (2147483647, 65535, 65535).

A Figura 3 representa o esquemático da configuração da arquitetura da NVIDIA Tesla T4.

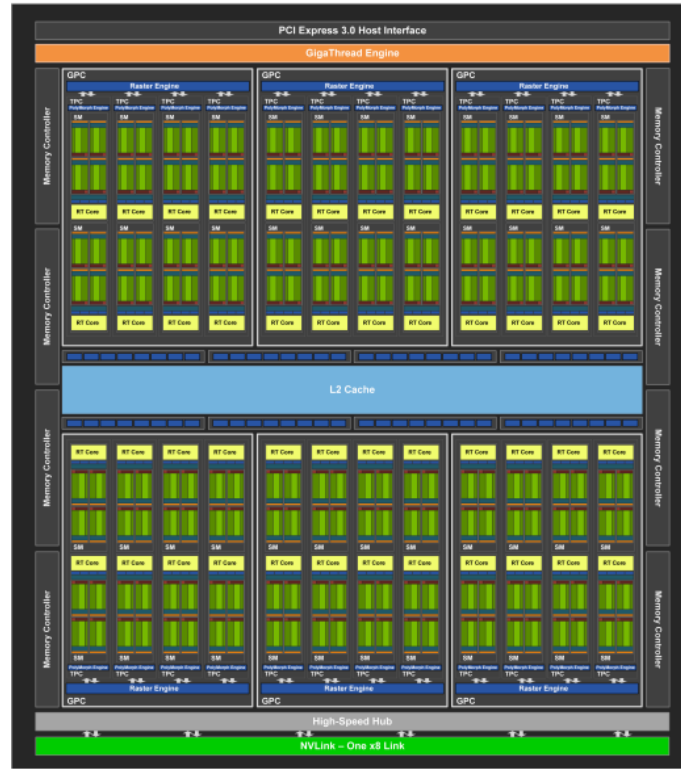


Figura 3. Arquitetura da GPU Testa T4. [NVIDIA Corporation 2018]

3. Desenvolvimento

A simulação de um modelo de difusão de contaminantes em água pode ser equacionado por:

$$\frac{\partial C}{\partial t} + \mathbf{v} \cdot \nabla C = D \nabla^2 C$$

Onde:

- C é a concentração do contaminante.
- t é o tempo (em segundos).
- $\mathbf{v}$ é o vetor de velocidade do fluxo da água.
- D é o coeficiente de difusão do contaminante na água.
- $\nabla^2 C$ é a taxa de variação da concentração no espaço.

Ou ainda, pode ser formulada de uma modo mais extenso

$$C_{i,j}^{(t+1)} = C_{i,j}^{(t)} + D \cdot \Delta t \cdot \frac{C_{i+1,j}^{(t)} + C_{i-1,j}^{(t)} + C_{i,j+1}^{(t)} + C_{i,j-1}^{(t)} - 4 \cdot C_{i,j}^{(t)}}{\Delta x^2}$$

3.1. Paralelizando com o CUDA: Stencil 1D

O stencil 1D utiliza um vetor unidirecional para realizar os cálculos, tendo o cuidado de não acessar espaços de memória que não possam ser acessados.

```

1 double diff_eq(double *C, double *C_new) {
2     double *d_C, *d_C_new;
3     double *swap;
4
5
6     cudaMalloc( (void **) &d_C, SIZE);
7     cudaMalloc( (void **) &d_C_new, SIZE);
8
9
10    cudaMemcpy(d_C, C, SIZE, cudaMemcpyHostToDevice );
11    cudaMemcpy(d_C_new, C_new, SIZE, cudaMemcpyHostToDevice );
12
13
14    double difmedio;
15
16
17    for (int t = 0; t < T; t++) {
18        diff_eq_kernel<<< (N * N + (THREADS_PER_BLOCK-1)) / THREADS_PER_BLOCK, THREADS_PER_BLOCK >>>(d_C, d_C_new, N);
19
20
21        cudaDeviceSynchronize();
22
23
24        swap = d_C;
25        d_C = d_C_new;
26        d_C_new = swap;
27
28
29        if (t%100==0) {
30            difmedio = .0;
31            cudaMemcpy(C, d_C, SIZE, cudaMemcpyDeviceToHost);
32            cudaMemcpy(C_new, d_C_new, SIZE, cudaMemcpyDeviceToHost);
33
34
35            for(int i=0; i<N*N; i++)
36                difmedio += fabs(C[i] - C_new[i]);
37
38
39            printf("interacao %d - diferenca=%g\n", t, difmedio/((N-2)*(N-2)));
40        }
41    }
42
43
44    cudaMemcpy(C, d_C, SIZE, cudaMemcpyDeviceToHost);
45
46
47    cudaFree(d_C);
48    cudaFree(d_C_new);
49
50
51    return C[(N/2) * N + (N/2)];
52 }

```

Listagem 2. Chamada do *kernel*.

No trecho de código mostrado na Figura acima, é onde ocorre o preparo das variáveis para entrar no kernel, com as funções do cuda, como `cudaMalloc` e `cudaMemcpy`. As variáveis chaves são `d_C`, que contém a concentração atual, `d_C_new`, que calcula a próxima concentração da água, `swap`, uma variável auxiliar para troca entre o `d_C` e `d_C_new`. Ainda há a variável `difmedio`, para calcular a concentração média.

É chamado a função do kernel, `diff_eq_kernel`, passando os parâmetros da quantidade de threads e a quantidade de blocos, além das variáveis para o cálculo da concentração e a dimensão da grade, dado que ela é um quadrado.

Após a chamada é feita a sincronização com os threads e a troca de ponteiros pela variável. Também é mostrado a concentração média quando a iteração é múltipla de 100, passando os resultados para as variáveis do Host.

Quando todas as interações acabarem, as variáveis do device são liberadas e a concentração do meio é retornada.

```

1 __global__ void diff_eq_kernel(double *C, double *C_new, int n) {
2     int in dex = threadIdx.x + blockIdx.x * blockDim.x;
3
4

```

```

5 | if (index >= n && index < n * (n - 1) && index % n != 0 && index % n != n - 1) {
6 |     C_new[index] = C[index] + D * DELTA_T * (
7 |         (C[index+1] + C[index-1] + C[index+N] + C[index-N] - 4*C[index]) / (DELTA_X * DELTA_X)
8 |     );
9 | }
10 | }

```

A Figura acima mostra a funcionalidade da função do kernel, que calcula a próxima concentração, utilizando apenas vetores unidirecionais, mas que representam a matriz, apenas estão agrupadas em uma única dimensão. Note que é calculado o índice da thread que fará o cálculo. Observe que as bordas da matriz não são calculadas, ou seja, o lado esquerdo não tem relação alguma com o lado direito, ou em cima não tem relação com a parte de baixo. Já a equação é a mesma utilizada no código sequencial.

3.2. Paralelizando com o CUDA: Stencil 2D

O stencil 2D já utiliza uma matriz para realizar os cálculos, também com o cuidado de não acessar espaços de memória que não possam ser acessados. Contudo, a diferença entre o stencil 1D e o 2D é que, nessa implementação, no stencil 2D utilizou-se duas dimensões para encontrar o índice da *thread*. Especificamente as dimensões x e y da matriz do grid.

A implementação da chamada do *kernel* é bem similar à do stencil 1D, com pequenas variações entre as implementações, já que todos os laços percorrerão em outro laço, formando assim uma complexidade maior de cálculos. A implementação e chamada do *kernel* pode ser vista na Listagem 3.

```

1 | int main() {
2 |     double *C, *C_new;
3 |     double *d_C, *d_C_new;
4 |     size_t size = MEM_SIZE_MATRIX;
5 |
6 |     // Alocar memoria no host
7 |     C = (double*)malloc(size);
8 |     C_new = (double*)malloc(size);
9 |
10 |    if (C == NULL || C_new == NULL) {
11 |        fprintf(stderr, "Memory allocation failed\n");
12 |        return 1;
13 |    }
14 |
15 |    // Inicializar matrizes
16 |    initialize_matrix(C, N);
17 |    initialize_matrix(C_new, N);
18 |
19 |    // Alocar memoria na GPU
20 |    check_cuda_error(
21 |        cudaMalloc((void**)&d_C, size), "Failed to allocate device memory for C");
22 |    check_cuda_error(
23 |        cudaMalloc((void**)&d_C_new, size), "Failed to allocate device memory for C_new");
24 |
25 |    // Copiar dados do host para a GPU
26 |    check_cuda_error(
27 |        cudaMemcpy(d_C, C, size, cudaMemcpyHostToDevice), "Failed to copy data from host to device");
28 |
29 |    // Configurar grade e blocos
30 |    dim3 block(BLOCK_SIZE_X, BLOCK_SIZE_Y);
31 |    dim3 grid((N + block.x - 1) / block.x, (N + block.y - 1) / block.y);
32 |
33 |    double difmedio;
34 |
35 |    // Calculando o tempo de execucao
36 |    double iStart = omp_get_wtime();
37 |
38 |    // Executar kernel
39 |    for (int t = 0; t < T; t++) {
40 |        diff_eq_kernel<<<grid, block>>>(d_C, d_C_new, N);
41 |        check_cuda_error(
42 |            cudaDeviceSynchronize(), "Kernel execution failed");
43 |
44 |        // Trocar ponteiros
45 |        double *swapAux = d_C;
46 |        d_C = d_C_new;
47 |        d_C_new = swapAux;
48 |    }

```

```

49     if (t % 100 == 0) {
50         difmedio = 0.0;
51         check_cuda_error(
52             cudaMemcpy(C, d_C, size, cudaMemcpyDeviceToHost), "Failed to copy data from device to host");
53         check_cuda_error(
54             cudaMemcpy(C_new, d_C_new, size, cudaMemcpyDeviceToHost), "Failed to copy data from device to host");
55
56         for (int i = 0; i < N * N; i++)
57             difmedio += fabs(C[i] - C_new[i]);
58
59         printf("interacao %d - diferenca=%g\n", t, difmedio / ((N - 2) * (N - 2)));
60     }
61 }
62
63 double iElaps = omp_get_wtime() - iStart;
64
65 // Metricas
66 printf("\n<<<%d,%d>>>");
67 printf("%lf\n", iElaps);
68
69 // Copiar resultado de volta para o host
70 check_cuda_error(
71     cudaMemcpy(C, d_C, size, cudaMemcpyDeviceToHost), "Failed to copy data from device to host");
72
73 // Liberar memoria na GPU
74 cudaFree(d_C);
75 cudaFree(d_C_new);
76
77 // Liberar memoria no host
78 free(C);
79 free(C_new);
80
81 return 0;
82 }

```

Listagem 3. Função principal com a chamada do *kernel* 2D.

Diante da Listagem 3, observa-se que, inicialmente, são alocados os ponteiros responsáveis por armazenar os valores das matrizes. Destaca-se a chamada da função `initialize_matrix`, que realiza a alocação de memória para as estruturas e inicializa o valor no centro da matriz `C`. Em seguida, a memória é alocada na GPU utilizando a função `cudaMalloc`, e os dados são transferidos do host para a GPU por meio da função `cudaMemcpy`.

Após essa etapa, configura-se a grade e os blocos utilizando a função `dim3`, uma estrutura que define as dimensões da grade e dos blocos. Por fim, um laço de repetição é executado para T iterações, em que o *kernel* `diff_eq_kernel` é invocado para calcular a difusão do contaminante na água. Em cada iteração, a concentração é calculada com base na técnica de programação Stencil. Ao término do laço, os dados são copiados da GPU para o host, e a memória alocada tanto na GPU quanto no host é liberada.

A Listagem 4 mostra a implementação do *kernel* que realiza os cálculos e também verifica os índices úteis para o stencil, já que deve-se descontar o raio do stencil para não acessar posições inválidas da matriz. Em contraste com o *kernel*, as Listagens 5 e 6 mostram as funções utilizadas para alocar e inicializar as matrizes e para verificar os erros de execução do CUDA, respectivamente.

```

1  __global__ void diff_eq_kernel(double *C, double *C_new, int n) {
2      int x = blockIdx.x * blockDim.x + threadIdx.x;
3      int y = blockIdx.y * blockDim.y + threadIdx.y;
4      int index = y * n + x;
5
6      if (x > 0 && x < n - 1 && y > 0 && y < n - 1) {
7          C_new[index] = C[index] + D * DELTA_T * (
8              (C[index + 1] + C[index - 1] + C[index + n] + C[index - n] - 4 * C[index]) / (DELTA_X * DELTA_X)
9          );
10     }
11 }

```

Listagem 4. *Kernel* 2D.


```

1 void initialize_matrix(double *m, int n) {
2     for (int i = 0; i < n; i++) {
3         for (int j = 0; j < n; j++) {
4             m[i * n + j] = 0.0;
5         }
6     }
7     m[(n / 2) * n + (n / 2)] = 1.0;
8 }

```

Listagem 5. Inicialização e alocação das matrizes.

```

1 void check_cuda_error(cudaError_t err, const char *msg) {
2     if (err != cudaSuccess) {
3         fprintf(stderr, "CUDA Error: %s: %s\n", msg, cudaGetErrorString(err));
4         exit(EXIT_FAILURE);
5     }
6 }

```

Listagem 6. Verificação de erros do CUDA.

Observe que, diferentemente do *Stencil* 1D desenvolvido no subcapítulo 3.1, o *kernel* criado para o *Stencil* 2D possui índices em função de duas dimensões, calculados a partir dos métodos `blockIdx.x`, `blockDim.x` e `threadIdx.x`, assim como dos métodos aplicados na dimensão `y`.

4. Resultados

Após as implementações realizadas e mostradas na seção 3, foram realizados testes de desempenho para avaliar a eficácia da paralelização do modelo criado. As Figuras 4 e 5 mostram os resultados obtidos para o *Stencil* 2D e 1D, respectivamente.

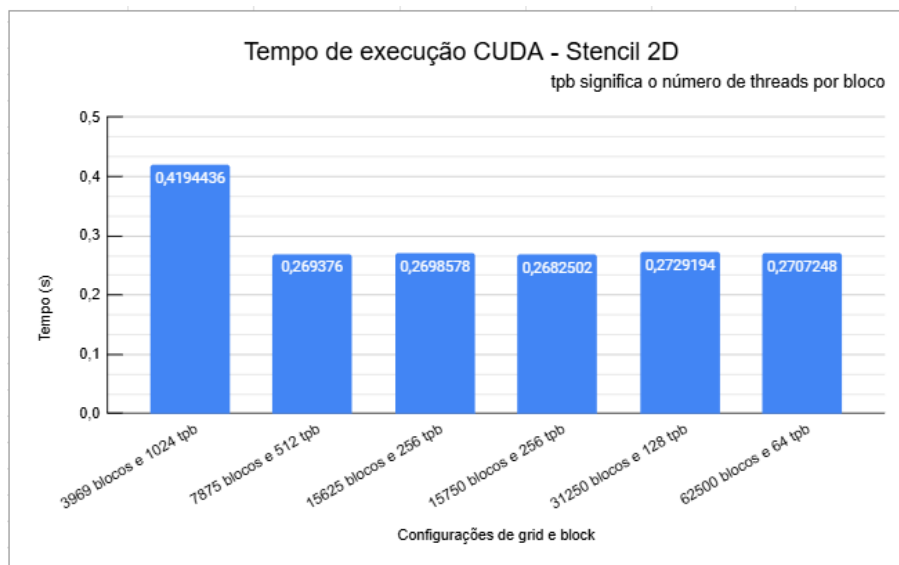


Figura 4. Gráfico de desempenho do *Stencil* 2D.

Diante os resultados das Figuras 4 e 5, pode-se verificar que os resultados para diversas configurações de *grids* e *blocks* foram satisfatórios, já que para um problema com um número expressivo de iterações, o tempo de processamento total está na ordem de milissegundos. Contudo, é possível observar que o *Stencil* 2D obteve um desempenho superior ao *Stencil* 1D, o que pode ser justificado pela complexidade do cálculo, que envolve mais operações matriciais. Ainda sobre os resultados do *stencil* 2D, também

pode-se observar que a configuração em que o número de blocos e o número de *threads* por bloco são de 7875 e 512, respectivamente.

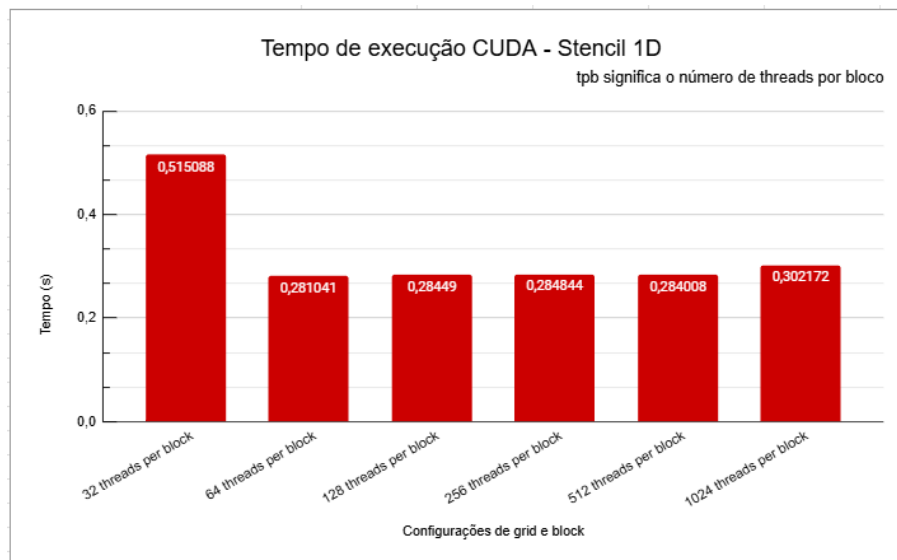


Figura 5. Gráfico de desempenho do *Stencil* 1D.

Já sobre o *stencil* 1D, o desempenho também foi satisfatório mas inferior ao *stencil* 2D, o que pode ser explicado pela forma como os dados são acessados na memória. A configuração que apresentou melhor desempenho foi com 16384 blocos e 256 *threads* por bloco (64 *threads* por bloco), resultando em um tempo de execução médio de 281 milissegundos.

Comparando as duas implementações, pode-se observar que ambas as abordagens atingiram resultados expressivos em termos de desempenho. O *stencil* 2D demonstrou ser mais eficiente para este problema específico, possivelmente devido à melhor utilização da localidade espacial da memória e ao padrão de acesso aos dados mais otimizado. Por outro lado, o *stencil* 1D, embora com tempos de execução ligeiramente superiores, ainda manteve um excelente desempenho, demonstrando que ambas as estratégias são viáveis para a resolução do problema.

A Tabela 1 apresenta uma comparação entre as melhores médias das amostras quando implementadas em código sequencial, OpenMP e CUDA. Deve-se levar em consideração a confiabilidade dos resultados, já que o número de testes realizados foi superior a 30. Diante os resultados obtidos, a implementação que melhor performou foi a codificação utilizando a tecnologia CUDA, com um *speedup* de 80.1603 em relação à implementação sequencial. Por fim, a eficiência da implementação em CUDA não foi calculada, já que a modelagem CUDA apresenta características únicas, quando comparadas às demais. Portanto, o cálculo da eficiência para tal arquitetura deve ser diferente e não foi considerado para esse trabalho.

5. Conclusão

Os resultados obtidos demonstram que a paralelização em GPU utilizando CUDA foi extremamente bem-sucedida, reduzindo significativamente o tempo de processamento em

Tabela 1. Comparação entre as diferentes implementações

Implementação	Tempo (ms)	Speedup	Eficiência
Sequencial	21.507	1.00	1.00
OpenMP	16.1707	1.3300	0.3325
CUDA 1D	0.2810	76.5374	-
CUDA 2D	0.2683	80.1603	-

comparação com uma possível implementação sequencial. Além disso, as diferentes configurações de blocos e *threads* testadas permitiram identificar as combinações ideais para cada implementação, otimizando ainda mais o desempenho do algoritmo.

Este trabalho apresentou a implementação e análise de diferentes abordagens para a simulação da difusão de contaminantes em água, utilizando técnicas de programação paralela em CPU (OpenMP) e GPU (CUDA). Os resultados obtidos demonstraram que a paralelização em GPU utilizando CUDA foi extremamente bem-sucedida, alcançando um speedup de até 80.1603 vezes em relação à implementação sequencial.

A comparação entre as implementações 1D e 2D em CUDA revelou que ambas as abordagens são eficientes, com a implementação 2D apresentando um desempenho ligeiramente superior. Esta diferença pode ser atribuída à melhor utilização da localidade espacial da memória e ao padrão de acesso aos dados mais otimizado na versão 2D.

Por fim, é importante ressaltar que a implementação OpenMP, embora tenha apresentado uma melhoria de desempenho mais modesta (speedup de 1.33), também contribuiu para a redução do tempo de processamento. No entanto, a tecnologia CUDA demonstrou ser a escolha mais adequada para este tipo de problema, devido à sua capacidade de explorar o alto grau de paralelismo oferecido pelas GPUs.

6. Referências

References

- Cheng, J., Grossman, M., and McKercher, T. (2014). *Professional CUDA C Programming*. John Wiley and Sons, Inc.
- McKennon, J. (2013). Cuda parallel thread management.
- NVIDIA Corporation (2012). *CUDA C PROGRAMMING GUIDE*.
- NVIDIA Corporation (2018). *NVIDIA TURING GPU ARCHITECTURE*. NVIDIA.
- Owens, J. D., Houston, M., Luebke, D., Green, S., Stone, J. E., and Phillips, J. C. (2008). Gpu computing. *Proceedings of the IEEE*, 96(5):879–899. Retrieved from <http://dx.doi.org/10.1109/JPROC.2008.917757>.
- Raúl de la Cruz and Araya-Polo, M. (2015). Modeling stencil computations on modern hpc architectures. *Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS14)*.
- Stein, C. and Griebler, D. (2018). *Programação Paralela para GPU em Aplicações de Processamento de Stream*. PhD thesis.